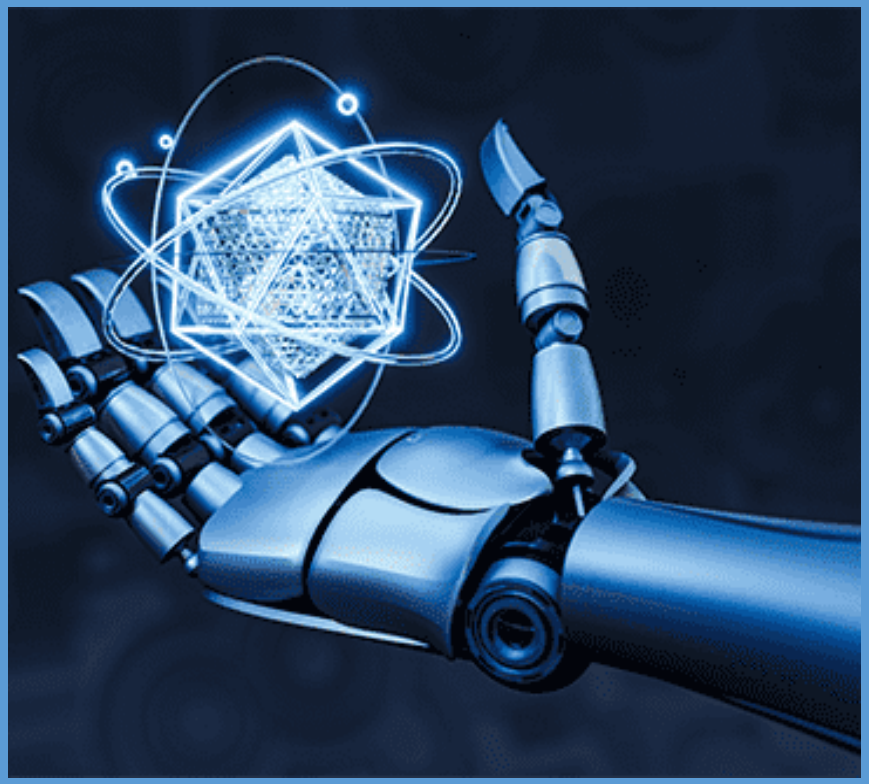
Applied AI & Machine Learning
CS-333

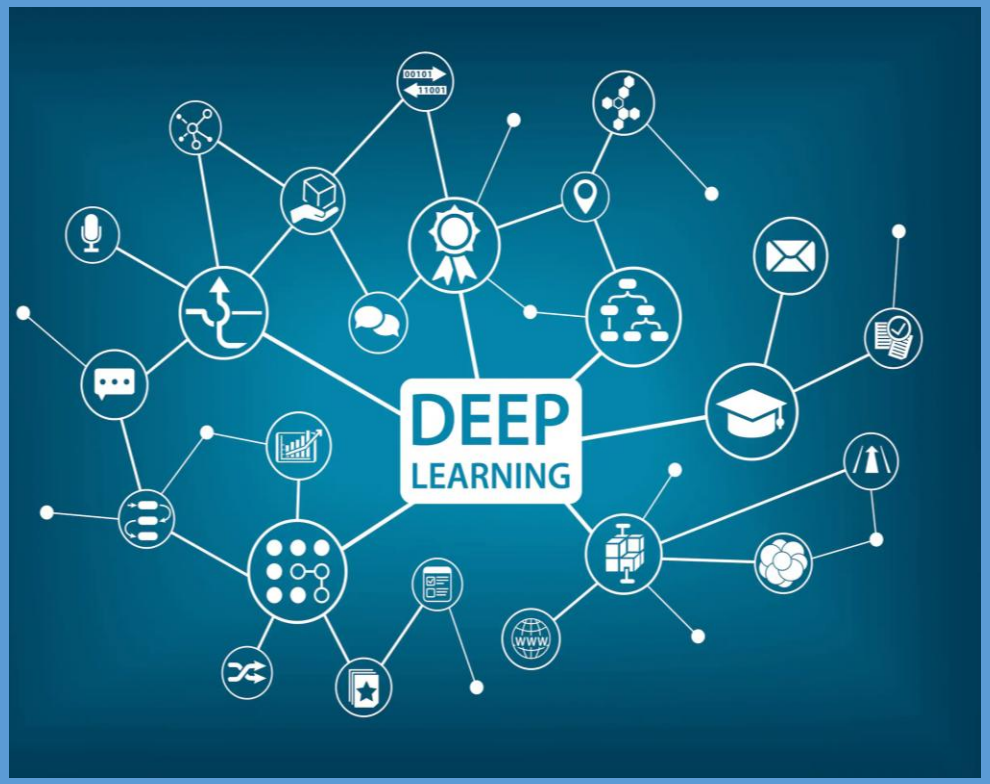
Dr. Abbas Hussain

PNEC, NUST

Lecture 10

Spring 2026





Convolutional Neural Networks (CNN)

Learning Outcomes

By the end of this lecture, students will be able to:

1. Understand the limitations of FCNN for image data
2. Explain the concept of spatial information in images
3. Describe the architecture and working of CNNs
4. Apply the convolution operation and compute feature maps
5. Interpret the role of filters (kernels) in feature extraction
6. Analyze how padding and stride affect output feature map size
7. Understand the role of activation functions (ReLU) in CNNs

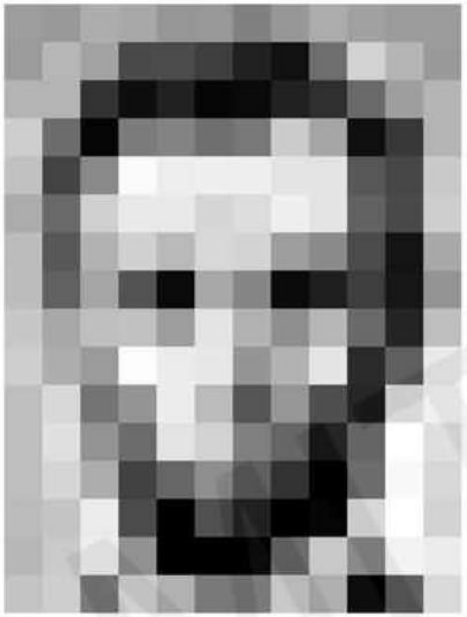
Topics

1. Visual Features
2. Fully Connected Neural Network
3. Convolutional Neural Networks (CNNs)
4. Convolution Operation
5. Activation Functions (ReLU)
6. Pooling Layers
7. Flattening and Fully Connected Layers

Lecture slides are taken from MIT 6.S191 Intro to deep learning course

Learning Visual Features

Images are Numbers



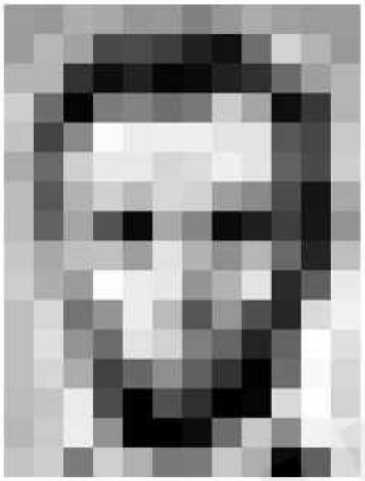
157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

What the computer sees

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

An image is just a matrix of numbers [0,255]!
i.e., 1080x1080x3 for an RGB image

Tasks in Computer Vision



Input Image

167	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Pixel Representation

classification

Lincoln	0.8
Washington	0.1
Jefferson	0.05
Obama	0.05

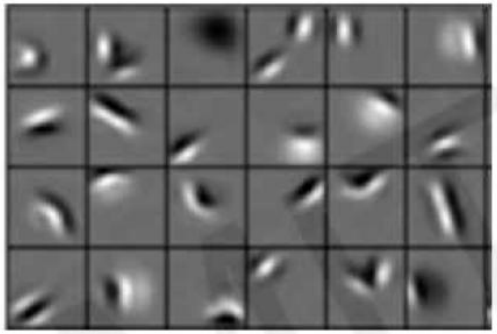
- **Regression**: output variable takes continuous value
- **Classification**: output variable takes class label. Can produce probability of belonging to a particular class

MIT 6.191

Learning Feature Representations

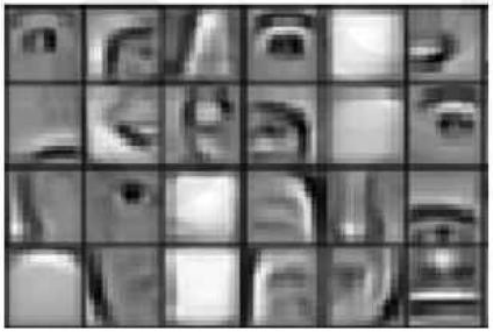
Can we learn a **hierarchy of features** directly from the data instead of hand engineering?

Low level features



Edges, dark spots

Mid level features



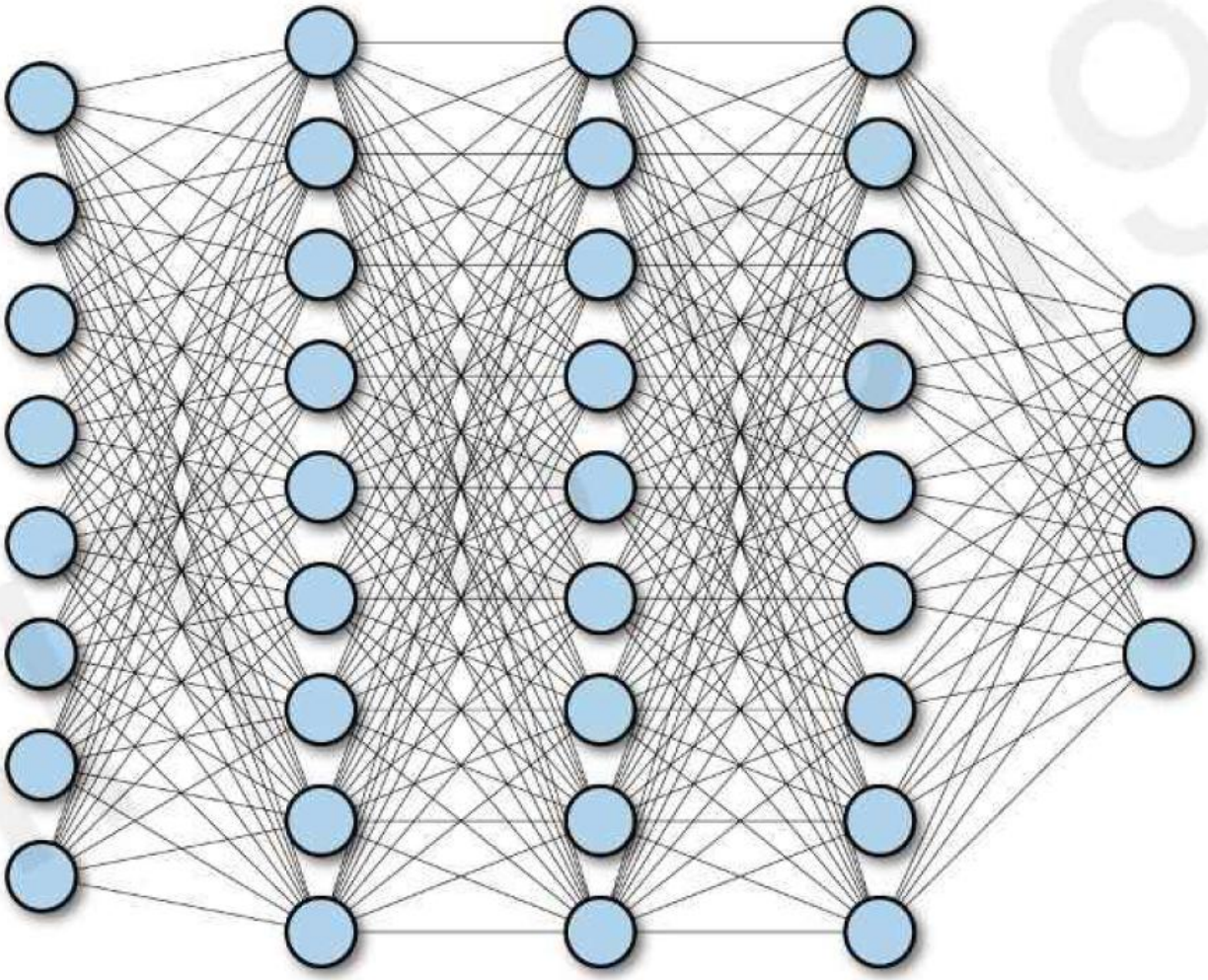
Eyes, ears, nose

High level features

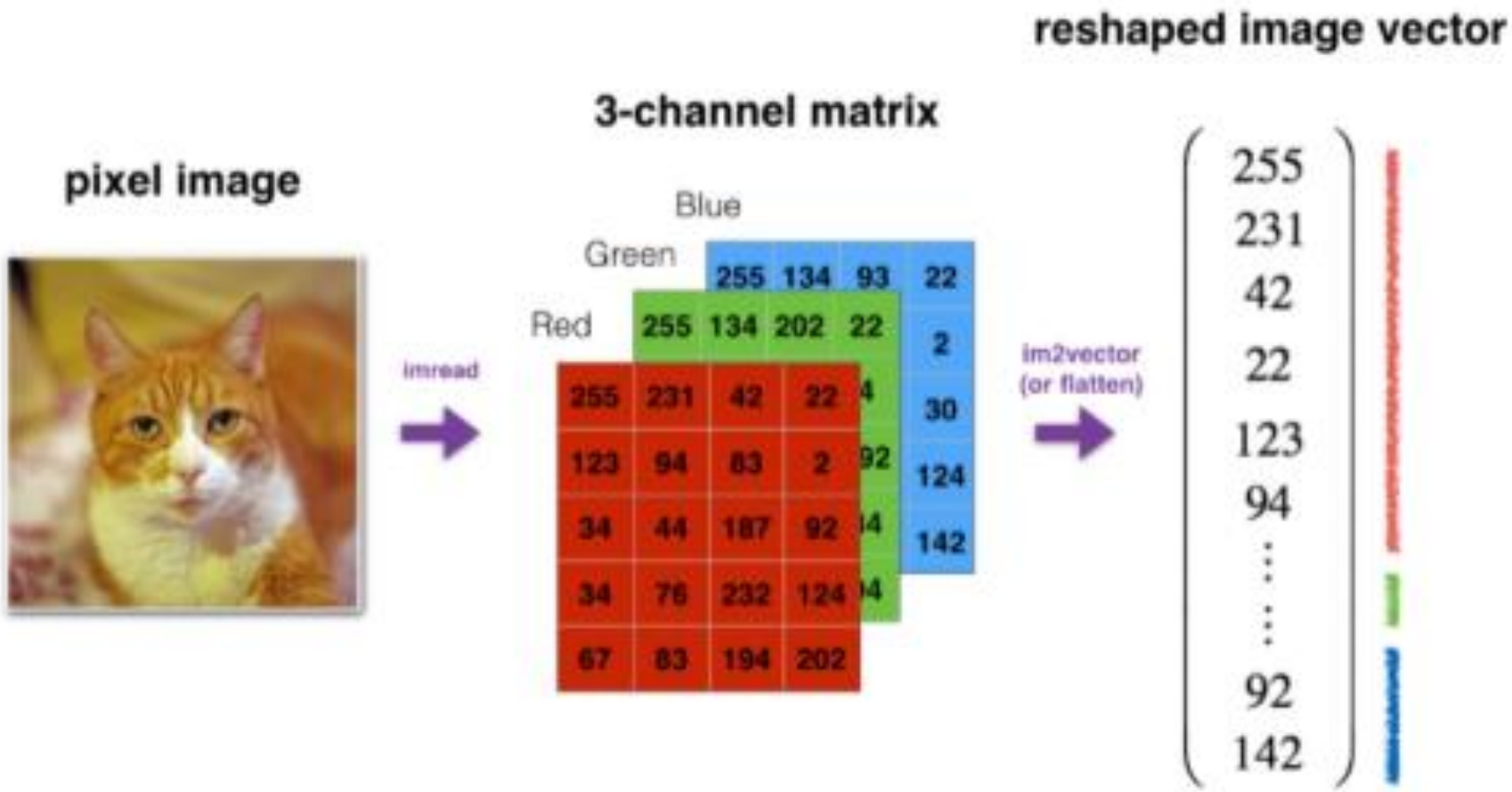
Facial structure

Fully Connected Neural Network

Fully Connected Neural Network



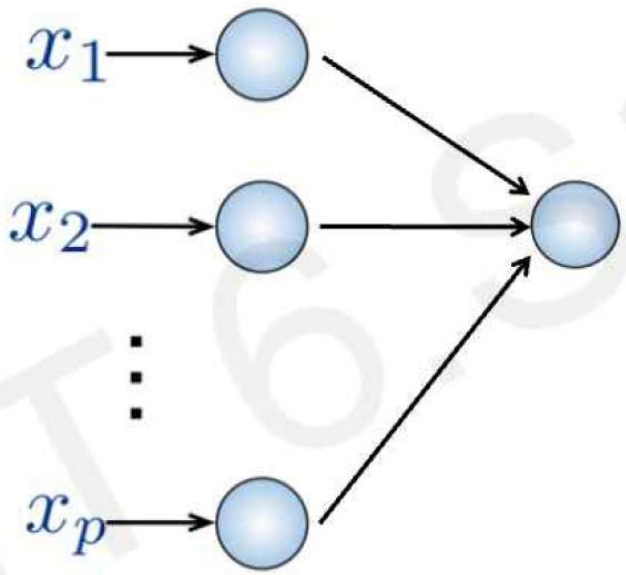
Fully Connected Neural Network



Fully Connected Neural Network

Input:

- 2D image
- Vector of pixel values



Fully Connected:

- Connect neuron in hidden layer to all neurons in input layer
- No spatial information!
- And many, many parameters!

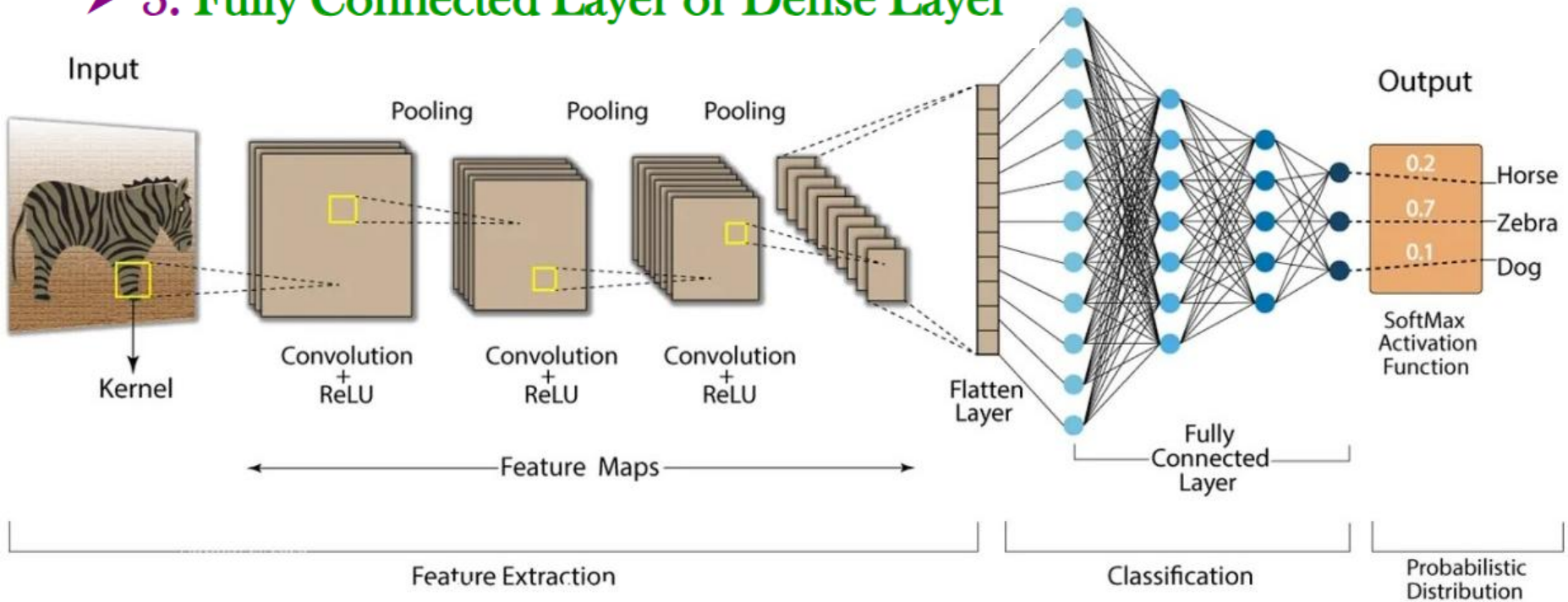
How can we use **spatial structure** in the input to inform the architecture of the network?

MIT 6.S191

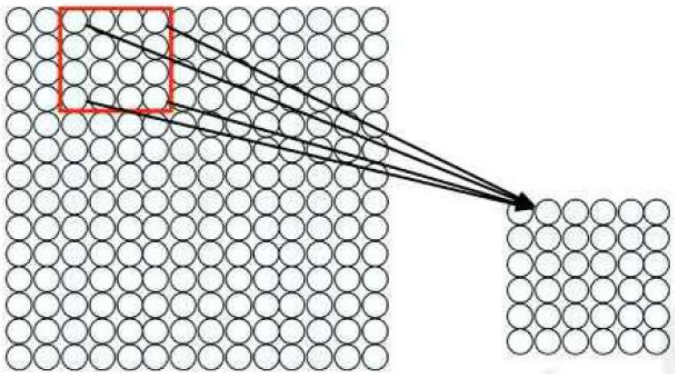
CNN (ConvNet)

Architecture

- CNN consists of 3 Layers. Those are
  - 1. Convolution Layer
  - 2. Pooling Layer
  - 3. Fully Connected Layer or Dense Layer



Feature Extraction with Convolution



- Filter of size 4x4 : 16 different weights
- Apply this same filter to 4x4 patches in input
- Shift by 2 pixels for next patch

This "patchy" operation is **convolution**

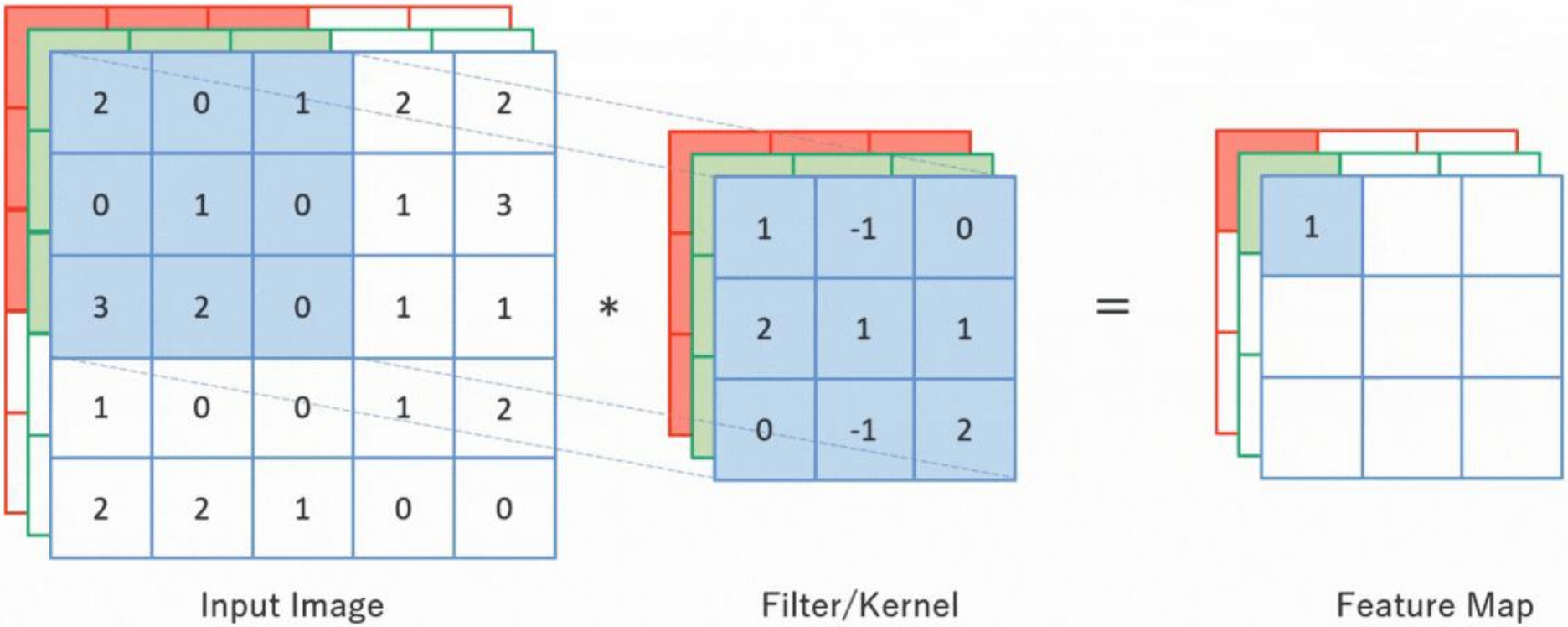
1) Apply a set of weights – a filter – to extract **local features**

2) Use **multiple filters** to extract different features

3) **Spatially share** parameters of each filter

MIT 6.S191

Convolutional Layer



The Convolution Operation

Suppose we want to compute the convolution of a 5x5 image and a 3x3 filter:

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

image

$\otimes$

1	0	1
0	1	0
1	0	1

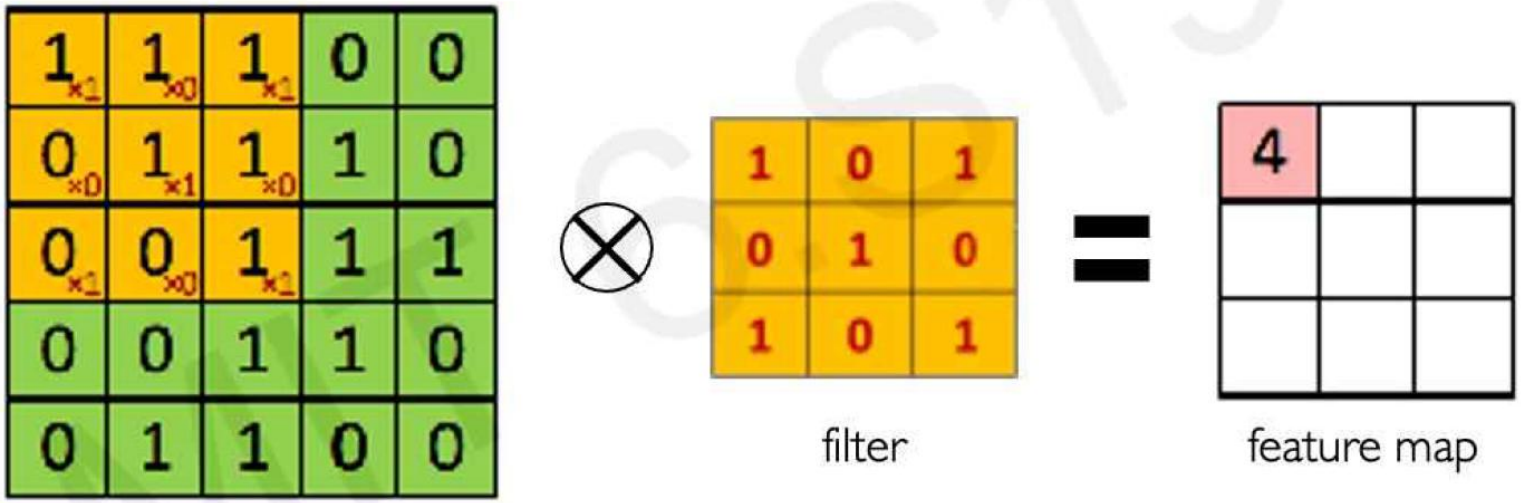
filter

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs…

MITeS191

The Convolution Operation

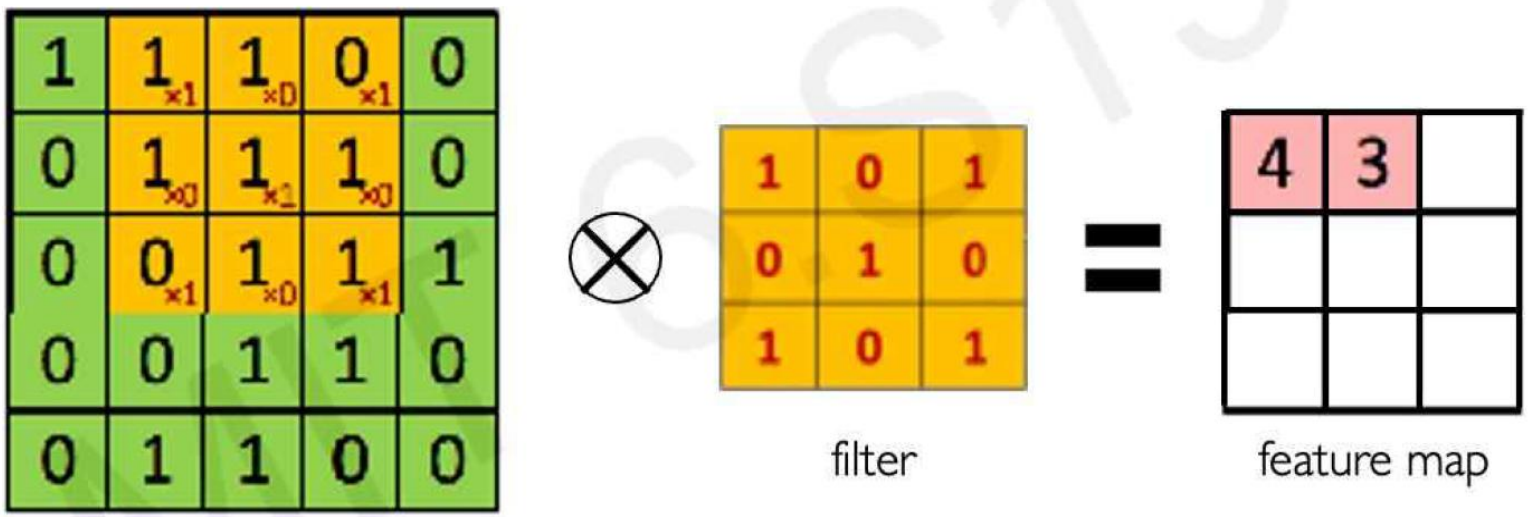
We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



[1x1 + 1x0 + 1x1 + 0x1 + 1x1+ 1x0 + 0x1 + 0x0 + 1x1] = 4

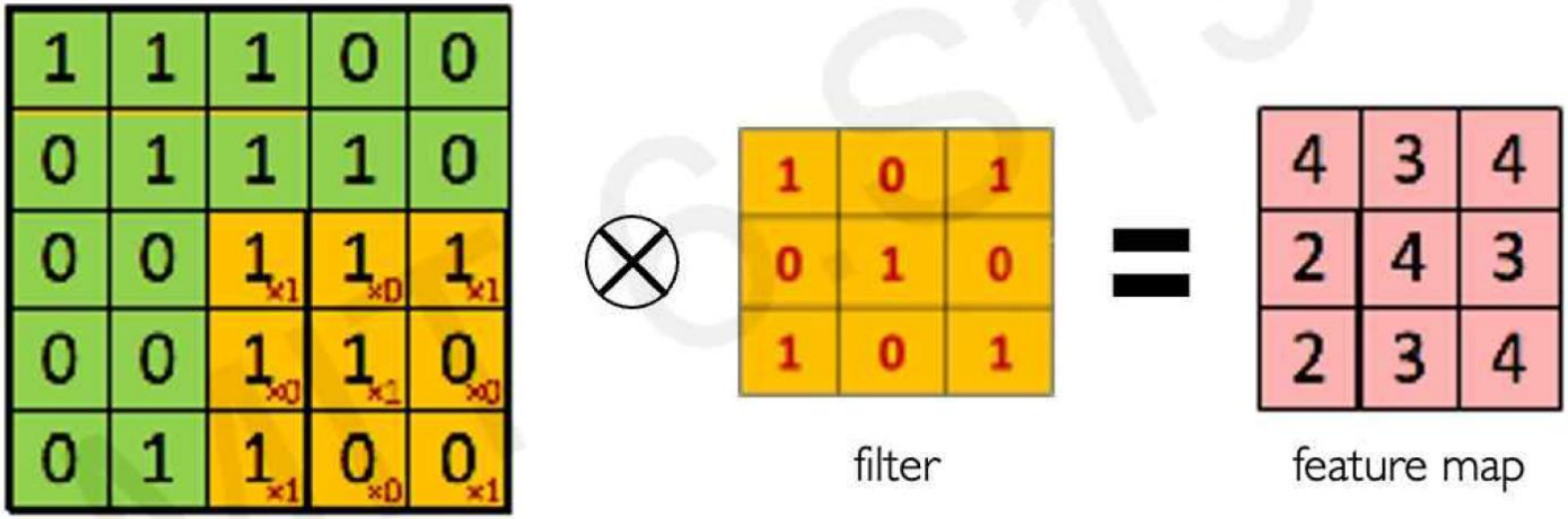
The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



Producing Feature Maps

Hand Engineered Filters

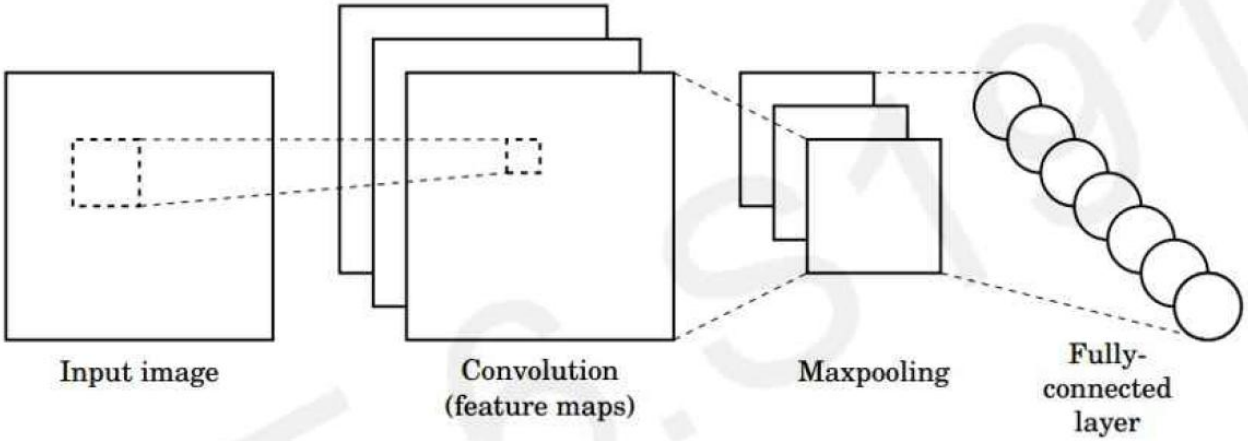
Original

Sharpen

Edge Detect

"Strong" Edge Detect

CNNs for Classification



`tf.keras.layers.Conv2D`	1. **Convolution**: Apply filters to generate feature maps.	`torch.nn.Conv2d`
`tf.keras.activations.*`	2. **Non-linearity**: Often ReLU.	`torch.nn.ReLU,…`
`tf.keras.layers.MaxPool2D`	3. **Pooling**: Downsampling operation on each feature map.	`torch.nn.MaxPool2d`

Train model with image data.
Learn weights of filters in convolutional layers.

Important Parameters in Convolutional Layer

- 1. Stride: Stride determines how many pixels the kernel shifts over the input at a time.

2. Padding: **Padding is a technique** used to preserve the spatial dimensions of the input image after convolution operations on a feature map. Padding involves adding extra pixels around the border of the input feature map before convolution.

Important Parameters in Convolutional Layer

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	5	4	3	2	1	0	0
0	0	5	5	3	2	1	0	0
0	0	5	4	5	2	1	0	0
0	0	5	4	3	5	1	0	0
0	0	5	4	5	5	1	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Input Image(x)

$*$

1	1	1
0	0	0
1	1	1

$$\text{Output Size} = \frac{N - F + 2P}{S} + 1$$

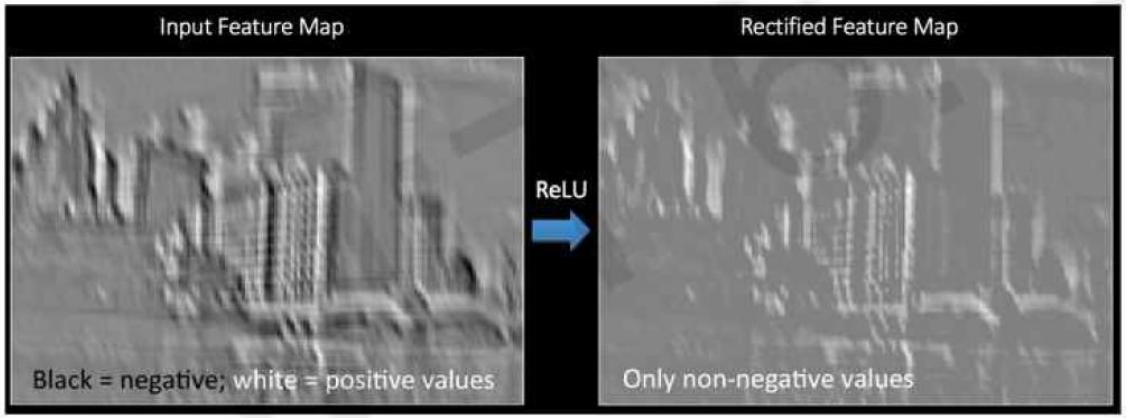
Where:

- N = Input size
- F = Filter size
- P = Padding
- S = Stride

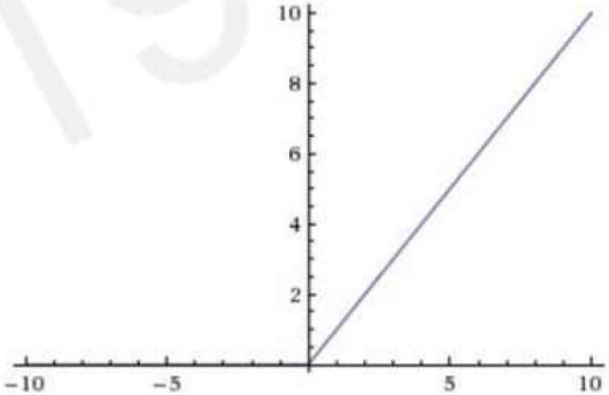
Padding

Introducing Non-Linearity

- Apply after every convolution operation (i.e., after convolutional layers)
- ReLU: pixel-by-pixel operation that replaces all negative values by zero. **Non-linear operation!**



Rectified Linear Unit (ReLU)

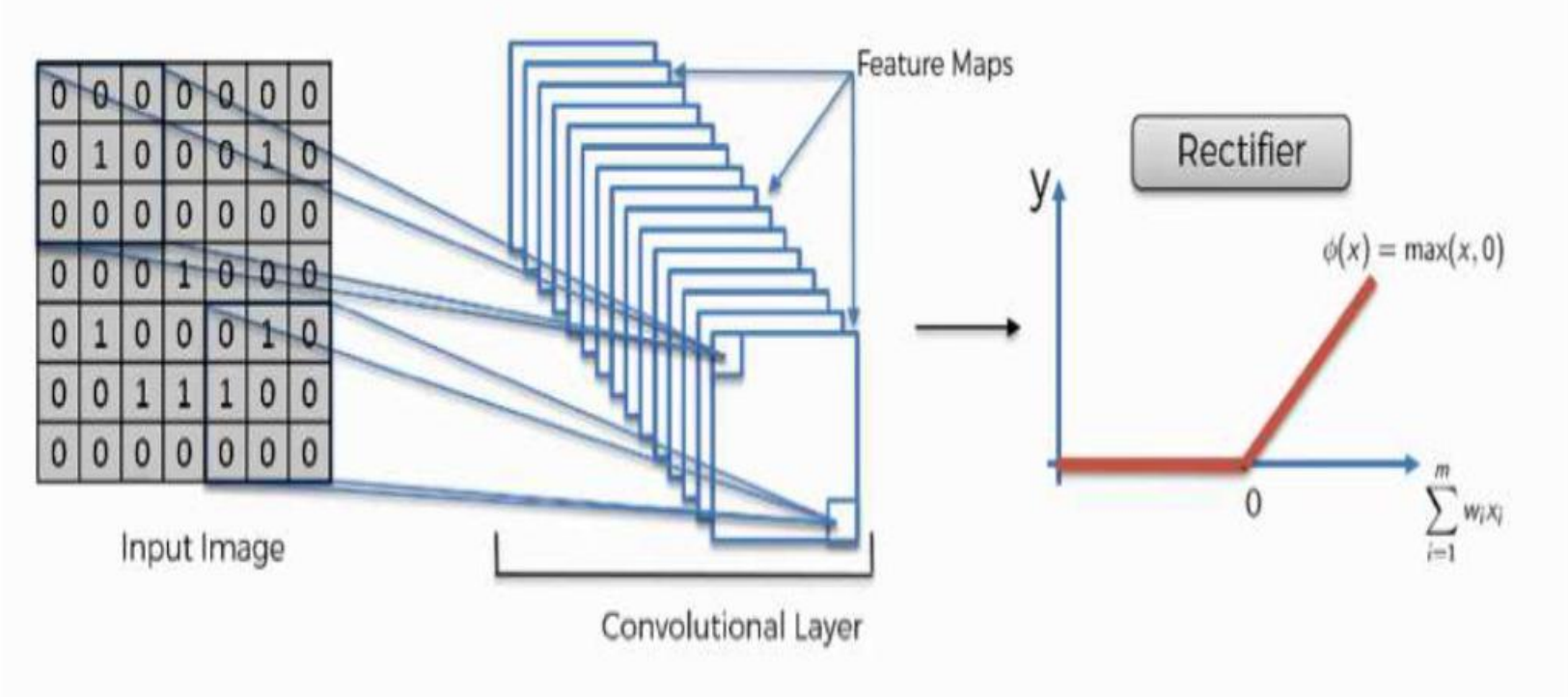


$$g(z) = \max(0, z)$$

`tf.keras.layers.ReLU`

`torch.nn.ReLU`

Introducing Non-Linearity



The purpose of applying the rectifier function is to increase the non-linearity in our images. The ReLU function takes a value x and returns 0 if x is negative and x if x is positive.

Introducing Non-Linearity

Input

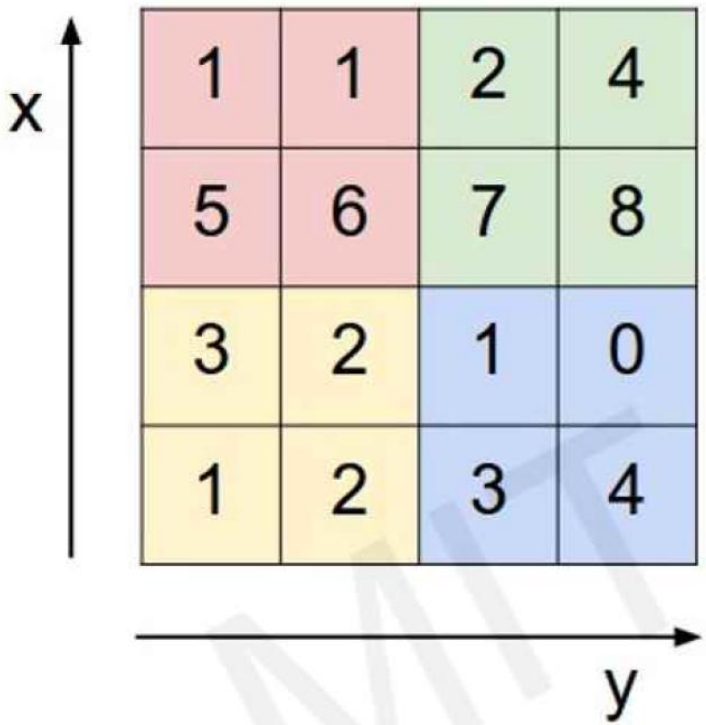
-249	-91	-37
250	-134	101
27	61	-153

ReLU

0	0	0
250	0	101
27	61	0

As you can see from the graph, the ReLU function is nonlinear. In this layer, the ReLU function is applied to each point in the feature map. The result is a feature map without negative values.

Pooling



max pool with 2x2 filters and stride 2

```
tf.keras.layers.MaxPool2D(
    pool_size=(2,2),
    strides=2
)
```

```
torch.nn.MaxPool2d(
    kernel_size=(2,2),
    stride=2
)
```

6	8
3	4

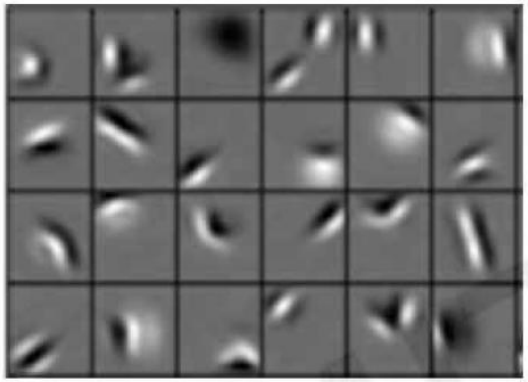
1) Reduced dimensionality
2) Spatial invariance

How else can we downsample and preserve spatial invariance?

MIT 6.S191

Representation Learning in Deep CNNs

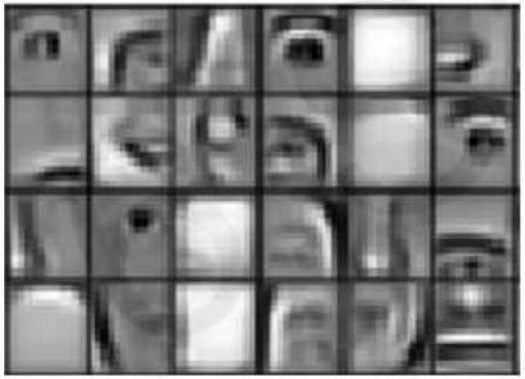
Low level features



Edges, dark spots

Conv Layer 1

Mid level features



Eyes, ears, nose

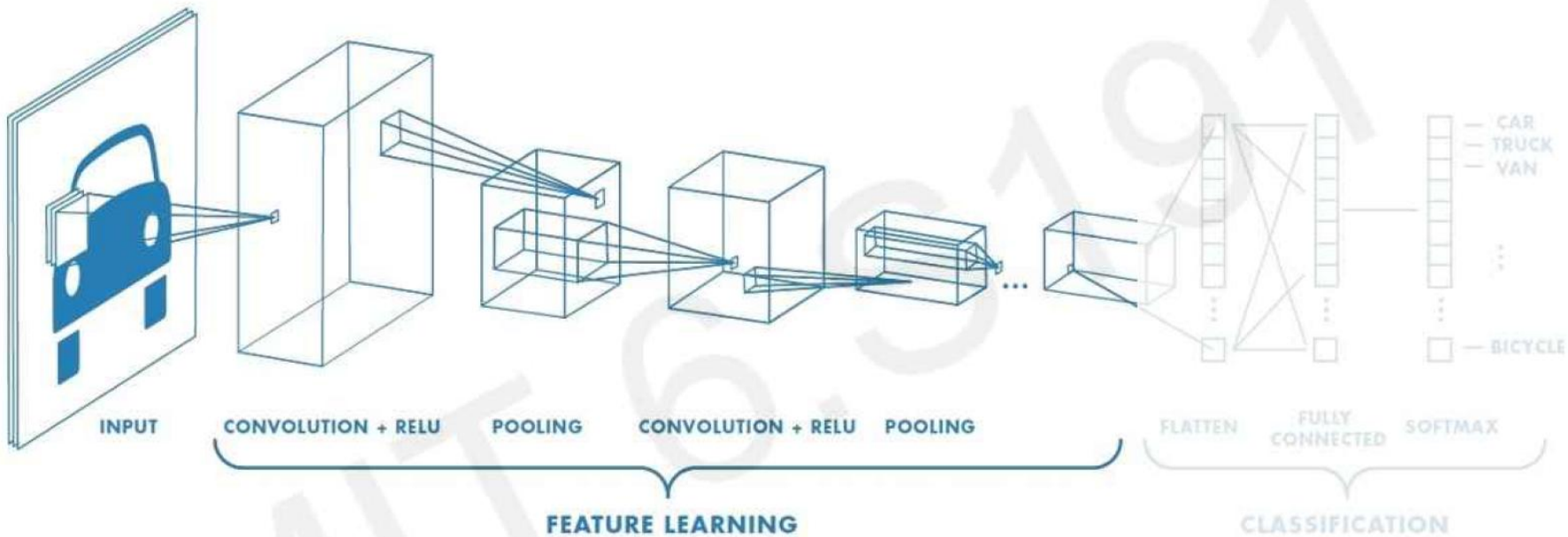
Conv Layer 2

High level features

Facial structure

Conv Layer 3

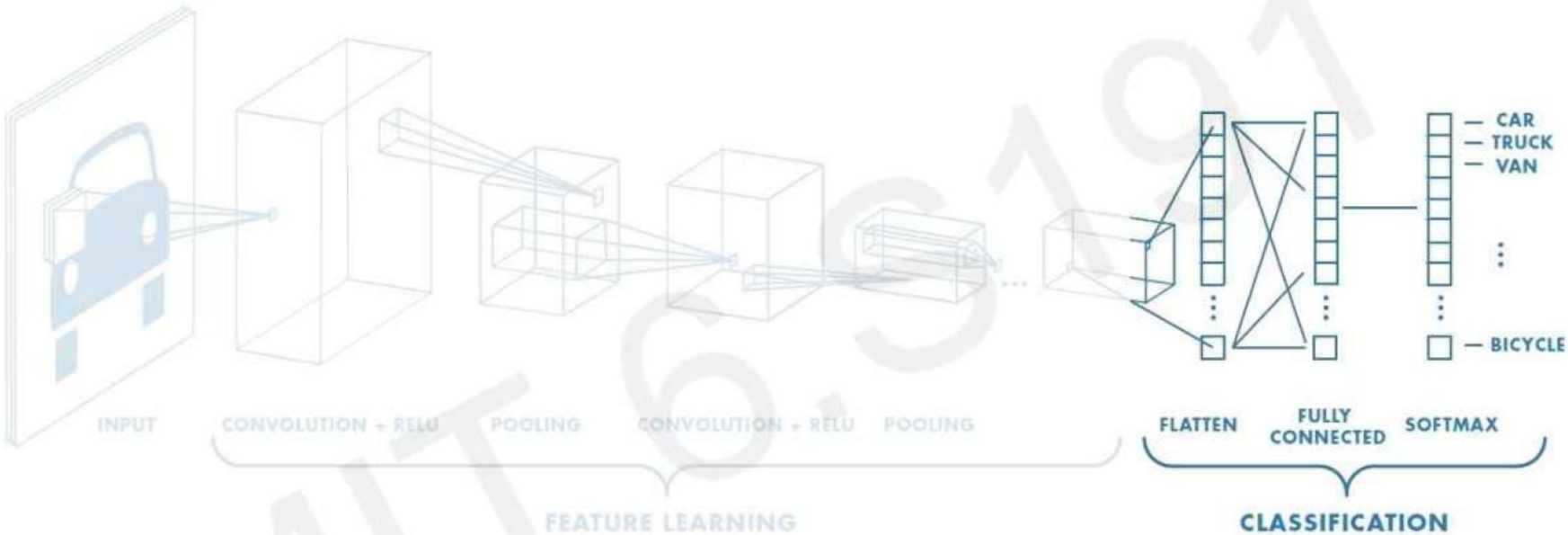
CNNs for Classification: Feature Learning



1. Learn features in input image through **convolution**
2. Introduce **non-linearity** through activation function (real-world data is non-linear!)
3. Reduce dimensionality and preserve spatial invariance with **pooling**

MIT 6.S191

CNNs for Classification: Class Probabilities



- CONV and POOL layers output high-level features of input
- Fully connected layer uses these features for classifying input image
- Express output as **probability** of image belonging to a particular class

$$\text{softmax}(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$

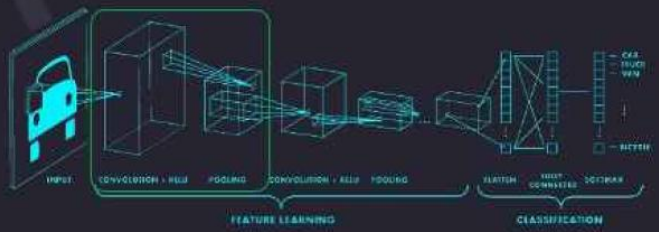
Putting it all together: CNN in TensorFlow

```python
import tensorflow as tf

def generate_model():
    model = tf.keras.Sequential([
        # first convolutional layer
        tf.keras.layers.Conv2D(32, filter_size=3, activation='relu'),
        tf.keras.layers.MaxPool2D(pool_size=2, strides=2),

        # second convolutional layer
        tf.keras.layers.Conv2D(64, filter_size=3, activation='relu'),
        tf.keras.layers.MaxPool2D(pool_size=2, strides=2),

        # fully connected classifier
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(1024, activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax')  # 10 outputs
    ])
    return model
```



MIT 6.S191

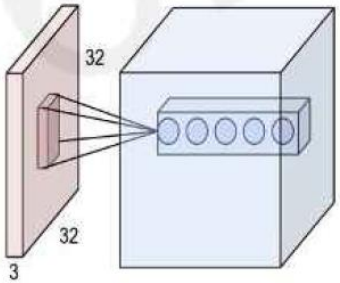
Deep Learning for Computer Vision: Summary

Foundations

- Why computer vision?
- Representing images
- Convolutions for feature extraction

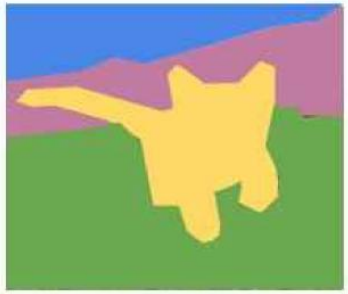
CNNs

- CNN architecture
- Application to classification
- ImageNet



Applications

- Segmentation, image captioning, control
- Security, medicine, robotics



MIT 6.S191